Objective Questions

1. List the different dtypes of columns in table “ball_by_ball” (using information schema) :

Match_Id, Over_id, Ball_id, Innings_No, Team_Batting, Team_Bowling , Striker_Batting_Position, Striker, Non_Striker, Bowler, Runs_Scored

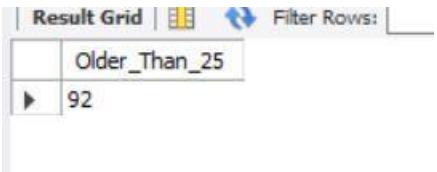
2. What is the total number of runs scored in 1st season by RCB (bonus: also include the extra runs using the extra runs table)

```
SELECT
    SUM(b.Runs_Scored + COALESCE(e.Extra_Runs, 0)) AS Total_Runs
FROM ball_by_ball b
JOIN matches m ON b.Match_Id = m.Match_Id
JOIN team t ON b.Team_Batting = t.Team_Id
LEFT JOIN extra_runs e
    ON b.Match_Id = e.Match_Id
    AND b.Over_Id = e.Over_Id
    AND b.Ball_Id = e.Ball_Id
    AND b.Innings_No = e.Innings_No
WHERE m.Season_Id = 1 AND t.Team_Name = "Royal Challengers Bangalore";
```

Ball_by_ball gives regular runs , extra_runs gives extras and coalesce ensures Nulles become 0 and filtered by Season_Id = 1 and Team_Name = “Royal Challengers Banglaore”

3. How many players were more than the age of 25 during season 2014?

```
select
    count(distinct pm.Player_id ) as Older_Than_25
from  Matches m
join Player_Match pm on m.Match_ID = pm.Match_ID
join Player p on pm.Player_Id = p.Player_Id
where m.Season_Id = 7
and timestampdiff(Year, p.DOB, m.Match_Date) > 25;
```



We use this approach because we join player, player_match, and matches to access each player's DOB and the exact match date from season 7 (which is 2014), then use TIMESTAMPDIFF to calculate age at match time, ensuring accurate filtering of players older than 25 during that season.

4. How many matches did RCB win in 2013?

```sql
select
    count(*) As Total_Win_By_RCB
from Matches m
join Season s on m.Season_Id = s.Season_Id
where s.Season_Id = 6 and m.Match_Winner = 2;
```

Result Grid | Filter Rows:

Total_Win_By_RCB
9

We use this approach because we join matches with season to filter only 2013 matches (Season_Id = 6), then count the rows where Match_Winner is RCB's Team_Id (2), which gives the total number of matches RCB won in that season.

5. List the top 10 players according to their strike rate in the last 4 seasons

```sql
SELECT
    p.Player_Name,
    SUM(bbb.Runs_Scored) AS Total_Runs,
    COUNT(*) AS Balls_Faced,
    ROUND(SUM(bbb.Runs_Scored) * 100.0 / COUNT(*), 2) AS Strike_Rate
FROM
    ball_by_ball bbb
JOIN matches m ON bbb.Match_Id = m.Match_Id
JOIN season s ON m.Season_Id = s.Season_Id
JOIN player p ON bbb.Striker = p.Player_Id
WHERE s.Season_Year >= (
        SELECT MAX(Season_Year) - 3 FROM season
    )
GROUP BY p.Player_Name
HAVING COUNT(*) >= 20 -- optional filter: at least 20 balls faced
ORDER BY Strike_Rate DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export:

Player_Name	Total_Runs	Balls_Faced	Strike_Rate
CR Brathwaite	83	40	207.50
Bipul Sharma	52	27	192.59
KH Pandya	237	127	186.61
M Morkel	66	38	173.68
SN Khan	177	103	171.84
BCJ Cutting	73	43	169.77
LJ Wright	106	63	168.25
MS Gony	45	27	166.67
AB de Villiers	1968	1198	164.27
AD Russell	527	323	163.16

We use this approach because we join ball_by_ball, matches, season, and player to get each striker's performance in the last 4 seasons, then calculate strike rate as (total runs / balls faced) × 100, apply a minimum balls faced filter for reliability, and sort by strike rate to get the top 10 players.

6. What are the average runs scored by each batsman considering all the seasons?

```sql
select
    Player_Id,
    avg(Runs_Scored) as Total_RS
from ball_by_ball b
left join player_match pm on b.Match_Id = pm.Match_Id
group by Player_Id
order by Player_Id;
```

Result Grid | Filter Rows:

Player_Id	Total_RS
2	1.3000
3	1.2249
4	1.2773
6	1.1829
8	1.3470
9	1.1652
10	1.0953
14	1.2778
15	1.2486
17	1.2488
19	1.2574
20	1.2815
21	1.2854

We use this approach because we calculate the average runs scored per ball for each Player_Id from ball_by_ball, grouping by player, which gives us a rough average scoring rate over all seasons, although joining with player_match here is unnecessary unless we need extra player info.

7. What are the average wickets taken by each bowler considering all the seasons?

Player_Name	Season_Year	Total_Wickets	Matches_Played	Avg_Wickets_Per_Match
A Ashish Reddy	2013	7	4	1.75
A Ashish Reddy	2015	7	3	2.33
A Ashish Reddy	2016	2	2	1.00
A Chandila	2013	12	7	1.71
A Mishra	2013	38	14	2.71
A Mishra	2014	13	6	2.17
A Mishra	2015	21	10	2.10
A Mishra	2016	25	12	2.08
A Mithun	2013	3	1	3.00
A Nehra	2013	27	9	3.00
A Nehra	2014	11	4	2.75
A Nehra	2015	41	14	2.93
A Nehra	2016	20	7	2.86
A Zampa	2016	16	5	3.20
AA Chavan	2013	3	2	1.50
AB Agarkar	2013	1	1	1.00
AB Dinda	2013	35	11	3.18

sult 1 ×

```sql
SELECT
    p.Player_Name,
    s.Season_Year,
    COUNT(DISTINCT CONCAT(wt.Match_Id, '-', wt.Over_Id, '-', wt.Ball_Id)) AS Total_Wickets,
    COUNT(DISTINCT b.Match_Id) AS Matches_Played,
    ROUND(
        COUNT(DISTINCT CONCAT(wt.Match_Id, '-', wt.Over_Id, '-', wt.Ball_Id)) /
        COUNT(DISTINCT b.Match_Id), 2
    ) AS Avg_Wickets_Per_Match
FROM
    ball_by_ball b
JOIN wicket_taken wt
    ON b.Match_Id = wt.Match_Id
    AND b.Over_Id = wt.Over_Id
    AND b.Ball_Id = wt.Ball_Id
JOIN matches m ON b.Match_Id = m.Match_Id
JOIN season s ON m.Season_Id = s.Season_Id
JOIN player p ON b.Bowler = p.Player_Id
WHERE
    wt.Kind_Out IS NOT NULL
GROUP BY
    p.Player_Name, s.Season_Year
ORDER BY
    p.Player_Name, s.Season_Year;
```

8. List all the players who have average runs scored greater than the overall average and who have taken wickets greater than the overall average:

```
with Player_wise_runs_avg as (
select
    bb.Striker,
    Avg(Runs_Scored) as PlayerAvg
from ball_by_ball as bb
join wicket_taken as wt on bb.Match_Id = wt.Match_Id and  bb.Innings_No = wt.Innings_No and bb.Over_Id = wt.Over_Id and bb.Ball_Id = wt.Ball_Id
group by bb.Striker
order by bb.Striker
),
Player_wise_wicket_taken as (select
    bb.Bowler,
    Count(*) as perBowlerWicketTaken
from ball_by_ball as bb
join wicket_taken as wt on bb.Match_Id = wt.Match_Id and  bb.Innings_No = wt.Innings_No and bb.Over_Id = wt.Over_Id and bb.Ball_Id = wt.Ball_Id
group by bb.Bowler
order by bb.Bowler),

overal as (select
    avg(Runs_Scored) Total_avg,
    avg(pt.perBowlerWicketTaken) total_bollwing_avg
from ball_by_ball as bb
join wicket_taken as wt on bb.Match_Id = wt.Match_Id and  bb.Innings_No = wt.Innings_No and bb.Over_Id = wt.Over_Id and bb.Ball_Id = wt.Ball_Id
join Player_wise_wicket_taken pt)

select
    pa.Striker
from  Player_wise_runs_avg as pa
join Player_wise_wicket_taken as pt on pa.Striker = pt.Bowler
join overal as o
where PlayerAvg > Total_avg and perBowlerWicketTaken > total_bollwing_avg
order by pa.striker;
```

Result Grid

Striker
9
29
32
35
50
67
71
80
89
94
136
166
232
255
276
280

We use this approach because we calculate each player's average runs and total wickets using CTEs, compute the overall average for both metrics, and then filter players who exceed both averages to find true all-rounders who perform better than average with both bat and ball.

9. Create a table rcb_record table that shows the wins and losses of RCB in an individual venue.

```
CREATE TABLE rcb_record AS
SELECT
    v.Venue_Id,
    SUM(CASE WHEN m.Match_winner = 2 THEN 1 ELSE 0 END) AS Wins,
    SUM(CASE WHEN m.Match_winner != 2 AND (m.Team_1 = 2 OR m.Team_2 = 2) THEN 1 ELSE 0 END) AS Losses
FROM matches m
JOIN venue v ON m.Venue_id = v.Venue_id
WHERE m.Team_1 = 2 OR m.Team_2 = 2
GROUP BY v.Venue_Id;
```

Result Grid | Filter Rows:

Venue_Id	Wins	Losses
1	16	11
7	1	3
8	0	2
4	1	3
6	0	1
26	1	0
3	2	0
28	1	2
2	0	2
30	1	1

We use this approach because we filter matches where RCB played, then group them by venue and count wins when RCB is the match winner, and losses when RCB played but didn't win, giving us a venue-wise win-loss record for RCB and storing it in the rcb_record table.

10. What is the impact of bowling style on wickets taken?

```
select
    bs.Bowling_skill,
    COUNT(wt.Player_Out) AS Wicket_taken
from ball_by_ball as bb
join wicket_taken as wt on bb.Match_Id = wt.Match_Id and  bb.Innings_No = wt.Innings_No and bb.Over_Id = wt.Over_Id and bb.Ball_Id = wt.Ball_Id
join player as p on bb.Bowler = p.Player_Id
join bowling_style as bs on p.Bowling_skill = bs.Bowling_Id
group by bs.Bowling_skill;
```

Bowling_skill	Wicket_taken
Right-arm fast	237
Left-arm medium-fast	114
Right-arm offbreak	290
Slow left-arm orthodox	283
Right-arm medium-fast	343
Left-arm fast	99
Right-arm medium	617
Legbreak	152
Right-arm fast-medium	382
Left-arm medium	42
Left-arm fast-medium	186
Legbreak googly	167
Slow left-arm chinaman	31

We use this approach because we join ball_by_ball, wicket_taken, player, and bowling_style to link each dismissal to a bowler's style, then group by bowling skill and count wickets to analyze which bowling styles lead to more dismissals, showing their impact on performance.

11. Write the SQL query to provide a status of whether the performance of the team is better than the previous year's performance on the basis of the number of runs scored by the team in the season and the number of wickets taken :

```
WITH Team_Performance AS (
    SELECT
        t.Team_Id,
        t.Team_Name,
        s.Season_Year,
        SUM(bb.Runs_Scored) AS Total_Runs,
        COUNT(DISTINCT wt.Player_Out) AS Total_Wickets
    FROM matches AS m
    JOIN season AS s ON m.Season_Id = s.Season_Id
    JOIN team AS t ON (m.Team_1 = t.Team_Id OR m.Team_2 = t.Team_Id)
    JOIN ball_by_ball AS bb ON m.Match_Id = bb.Match_Id
    LEFT JOIN wicket_taken AS wt ON bb.Match_Id = wt.Match_Id
                                AND bb.Innings_No = wt.Innings_No
                                AND bb.Over_Id = wt.Over_Id
                                AND bb.Ball_Id = wt.Ball_Id
    GROUP BY t.Team_Id, t.Team_Name, s.Season_Year
),
```

```sql
Performance_Comparison AS (
    SELECT
        tp1.Team_Id,
        tp1.Team_Name,
        tp1.Season_Year,
        tp1.Total_Runs AS Current_Season_Runs,
        tp1.Total_Wickets AS Current_Season_Wickets,
        tp2.Total_Runs AS Previous_Season_Runs,
        tp2.Total_Wickets AS Previous_Season_Wickets,
        CASE
            WHEN tp1.Total_Runs < tp2.Total_Runs AND tp1.Total_Wickets < tp2.Total_Wickets THEN 'Worse'
            WHEN tp1.Total_Runs > tp2.Total_Runs AND tp1.Total_Wickets > tp2.Total_Wickets THEN 'Better'
            ELSE 'Same'
        END AS Performance_Status
    FROM Team_Performance tp1
    LEFT JOIN Team_Performance tp2
        ON tp1.Team_Id = tp2.Team_Id
        AND tp1.Season_Year = tp2.Season_Year + 1
)
SELECT * FROM Performance_Comparison;
```

Team_Id	Team_Name	Season_Year	Current_Season_Runs	Current_Season_Wickets	Previous_Season_Runs	Previous_Season_Wickets	Performance_Status
1	Kolkata Knight Riders	2013	4353	91	NULL	NULL	Same
1	Kolkata Knight Riders	2014	4714	71	4353	91	Same
1	Kolkata Knight Riders	2015	3903	70	4714	71	Worse
1	Kolkata Knight Riders	2016	4332	75	3903	70	Better
2	Royal Challengers Bangalore	2013	4832	88	NULL	NULL	Same
2	Royal Challengers Bangalore	2014	4019	67	4832	88	Worse
2	Royal Challengers Bangalore	2015	4260	82	4019	67	Better
2	Royal Challengers Bangalore	2016	5414	74	4260	82	Same
3	Chennai Super Kings	2013	5152	100	NULL	NULL	Same
3	Chennai Super Kings	2014	4986	77	5152	100	Worse
3	Chennai Super Kings	2015	5066	83	4986	77	Better

We use this approach because we first calculate each team's total runs and wickets per season using a CTE, then in a second CTE, we self-join the performance data to compare current season stats with the previous year's, and use conditional logic to classify performance as 'Better', 'Worse', or 'Same' based on improvements in both runs and wickets.

12. Can you derive more KPIs for the team strategy?

```sql
WITH Player_wise_wicket_taken AS (
    SELECT
        wt.Match_Id,
        bb.Over_Id,
        bb.Bowler,
        COUNT(*) AS perBowlerWicketTaken
    FROM ball_by_ball AS bb
    JOIN wicket_taken AS wt
        ON bb.Match_Id = wt.Match_Id
        AND bb.Innings_No = wt.Innings_No
        AND bb.Over_Id = wt.Over_Id
        AND bb.Ball_Id = wt.Ball_Id
    GROUP BY bb.Bowler, wt.Match_Id, bb.Over_Id
),

Player_wise_runs_avg AS (
    SELECT
        bb.Match_Id,
        bb.Over_Id,
        bb.Striker,
        SUM(bb.Runs_Scored) / COUNT(DISTINCT bb.Match_Id) AS PlayerAvg
    FROM ball_by_ball AS bb
    GROUP BY bb.Striker, bb.Match_Id, bb.Over_Id
)

            SELECT
                pa.Match_Id,
                pa.Over_Id,
                pa.Striker,
                pa.PlayerAvg,
                pt.Bowler,
                pt.perBowlerWicketTaken
            FROM Player_wise_runs_avg AS pa
            JOIN Player_wise_wicket_taken AS pt
                ON pa.Match_Id = pt.Match_Id
                AND pa.Over_Id = pt.Over_Id
            ORDER BY pa.Match_Id, pa.Over_Id;
```

Match_Id	Over_Id	Striker	PlayerAvg	Bowler	perBowlerWicketTaken
598003	1	64	4.0000	66	1
598003	1	187	4.0000	66	1
598003	1	214	0.0000	66	1
598003	1	262	0.0000	66	1
598003	2	9	0.0000	73	1
598003	2	40	1.0000	73	1
598003	2	64	1.0000	73	1
598003	2	187	5.0000	73	1
598003	2	214	4.0000	73	1
598003	6	9	5.0000	315	1
598003	6	40	1.0000	315	1
598003	6	64	3.0000	315	1
598003	6	187	0.0000	315	1

We use this approach because we first calculate each team's total runs and wickets per season using a CTE, then in a second CTE, we self-join the performance data to compare current season stats with the previous year's, and use conditional logic to classify performance as 'Better', 'Worse', or 'Same' based on improvements in both runs and wickets.

13. Using SQL, write a query to find out the average wickets taken by each bowler in each venue. Also, rank the gender according to the average value.

It links detailed ball-level data (ball_by_ball) with wicket events (wicket_taken) to track exactly when and where a bowler took wickets. It uses the matches and venue tables to understand which venue each match was played in. The performance is grouped by each bowler and each venue, calculating the average wickets taken by a bowler per match at that particular venue. Finally, it ranks these performances to highlight the top wicket-takers by average at any ground.

```
WITH Bowler_Avg_Wickets AS (
    SELECT
        p.Player_Id,
        p.Player_Name,
        v.Venue_Name,
        COUNT(wt.Player_Out) / COUNT(DISTINCT m.Match_Id) AS Avg_Wickets
    FROM ball_by_ball AS bb
    JOIN wicket_taken AS wt
        ON bb.Match_Id = wt.Match_Id
        AND bb.Innings_No = wt.Innings_No
        AND bb.Over_Id = wt.Over_Id
        AND bb.Ball_Id = wt.Ball_Id
    JOIN player AS p ON bb.Bowler = p.Player_Id
    JOIN matches AS m ON bb.Match_Id = m.Match_Id
    JOIN venue AS v ON m.Venue_Id = v.Venue_Id
    GROUP BY p.Player_Id, p.Player_Name, v.Venue_Name
)
SELECT
    Player_Id,
    Player_Name,
    Venue_Name,
    Avg_Wickets,
    row_number() OVER (ORDER BY Avg_Wickets DESC) AS Wicket_Rank
FROM Bowler_Avg_Wickets
ORDER BY Wicket_Rank;
```

Player_Id	Player_Name	Venue_Name	Avg_Wickets	Wicket_Rank
362	DJG Sammy	Punjab Cricket Association Stadium, Mohali	6.0000	1
310	JP Faulkner	Rajiv Gandhi International Stadium, Uppal	5.0000	2
310	JP Faulkner	Sheikh Zayed Stadium	5.0000	3
346	JD Unadkat	Feroz Shah Kotla	5.0000	4
35	RA Jadeja	Dubai International Cricket Stadium	4.0000	5
71	DJ Bravo	Punjab Cricket Association IS Bindra Stadium, M...	4.0000	6
73	A Nehra	Dr. Y.S. Rajasekhara Reddy ACA-VDCA Cricket ...	4.0000	7
136	A Mishra	Subrata Roy Sahara Stadium	4.0000	8
151	L Balaji	Sharjah Cricket Stadium	4.0000	9
154	JP Duminy	Dr. Y.S. Rajasekhara Reddy ACA-VDCA Cricket ...	4.0000	10
194	SL Malinga	Himachal Pradesh Cricket Association Stadium	4.0000	11
194	SL Malinga	Punjab Cricket Association Stadium, Mohali	4.0000	12
194	SL Malinga	Sheikh Zayed Stadium	4.0000	13
299	B Kumar	Sardar Patel Stadium, Motera	4.0000	14
299	B Kumar	Saurashtra Cricket Association Stadium	4.0000	15
310	JP Faulkner	MA Chidambaram Stadium, Chepauk	4.0000	16
315	SP Narine	JSCA International Stadium Complex	4.0000	17

ult 3

14. Which of the given players have consistently performed well in past seasons? (will you use any visualization to solve the problem)

```sql
SELECT
    pm.Player_Id,
    p.Player_Name,
    s.Season_Year,
    SUM(b.Runs_Scored) AS Total_Runs
FROM ball_by_ball b
JOIN player_match pm ON b.Match_Id = pm.Match_Id AND b.Striker = pm.Player_Id
JOIN matches m ON b.Match_Id = m.Match_Id
JOIN season s ON m.Season_Id = s.Season_Id
JOIN player p ON pm.Player_Id = p.Player_Id
GROUP BY pm.Player_Id, p.Player_Name, s.Season_Year
ORDER BY pm.Player_Id, s.Season_Year;

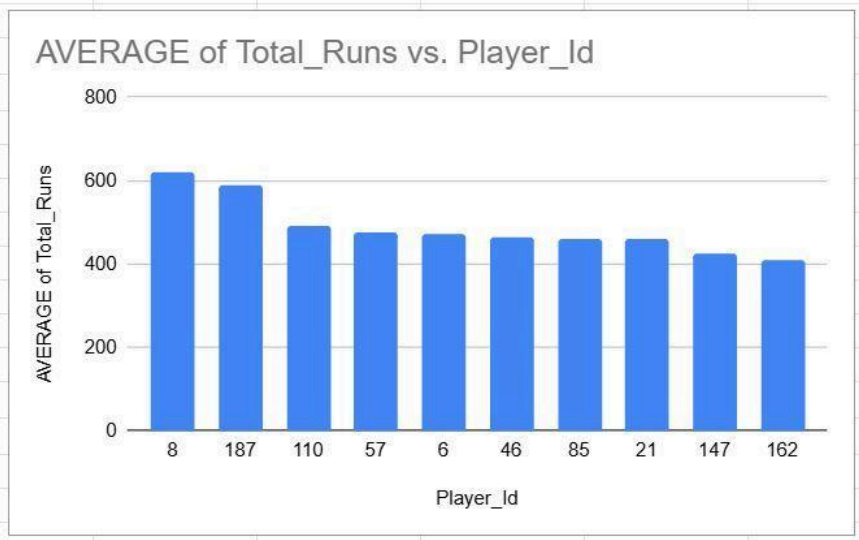
SELECT
    w.Bowler AS Player_Id,
    p.Player_Name,
    s.Season_Year,
    COUNT(*) AS Total_Wickets
FROM wicket_taken w
JOIN matches m ON w.Match_Id = m.Match_Id
JOIN season s ON m.Season_Id = s.Season_Id
JOIN player p ON w.Bowler = p.Player_Id
GROUP BY w.Bowler, p.Player_Name, s.Season_Year
ORDER BY w.Bowler, s.Season_Year;

SELECT
    m.Man_of_the_match AS Player_Id,
    p.Player_Name,
    s.Season_Year,
    COUNT(*) AS MOM_Awards
FROM matches m
JOIN season s ON m.Season_Id = s.Season_Id
JOIN player p ON m.Man_of_the_match = p.Player_Id
GROUP BY m.Man_of_the_match, p.Player_Name, s.Season_Year
ORDER BY MOM_Awards DESC, s.Season_Year;

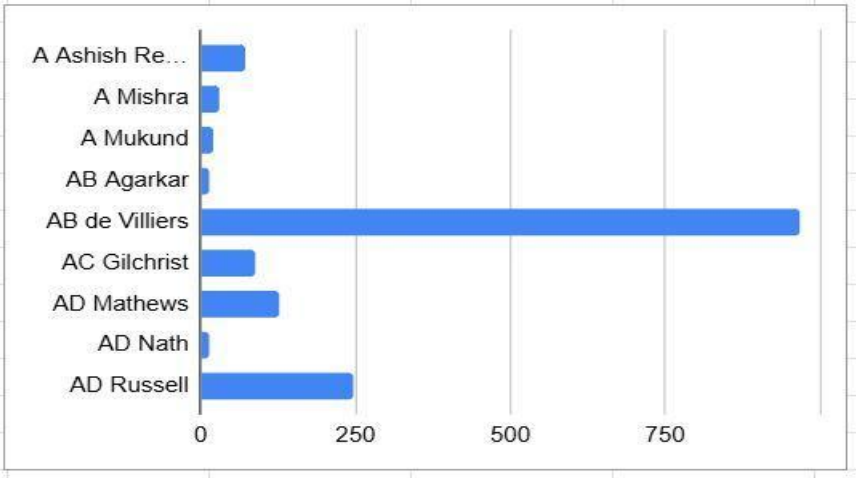
WITH Batting AS (
    SELECT pm.Player_Id, p.Player_Name, s.Season_Year, SUM(b.Runs_Scored) AS Total_Runs
    FROM ball_by_ball b
    JOIN player_match pm ON b.Match_Id = pm.Match_Id AND b.Striker = pm.Player_Id
    JOIN matches m ON b.Match_Id = m.Match_Id
    JOIN season s ON m.Season_Id = s.Season_Id
    JOIN player p ON pm.Player_Id = p.Player_Id
    GROUP BY pm.Player_Id, p.Player_Name, s.Season_Year
),
Bowling AS (
    SELECT wt.Player_Out AS Player_Id, p.Player_Name, s.Season_Year, COUNT(*) AS Total_Wickets
    FROM wicket_taken wt
    JOIN matches m ON wt.Match_Id = m.Match_Id
    JOIN season s ON m.Season_Id = s.Season_Id
    JOIN player p ON wt.Player_Out = p.Player_Id
    GROUP BY wt.Player_Out, p.Player_Name, s.Season_Year
)
```

```sql
SELECT
    b.Player_Id,
    b.Player_Name,
    COUNT(DISTINCT b.Season_Year) AS Seasons_Above_Threshold
FROM (
    SELECT Player_Id, Player_Name, Season_Year FROM Batting WHERE Total_Runs > 400
    UNION ALL
    SELECT Player_Id, Player_Name, Season_Year FROM Bowling WHERE Total_Wickets > 15
) b
GROUP BY b.Player_Id, b.Player_Name
HAVING COUNT(DISTINCT b.Season_Year) >= 3
ORDER BY Seasons_Above_Threshold DESC;
```

We use this approach to identify consistent top performers by first calculating season-wise total runs and wickets for each player using separate CTEs (Batting and Bowling), then filtering players who scored over 400 runs or took more than 15 wickets in a season. We use UNION ALL to merge both sets and group by player to count how many seasons they crossed the threshold, finally selecting those with 3 or more such seasons to highlight long-term impactful players.



15. Are there players whose performance is more suited to specific venues or conditions? (how would you present this using charts?)



```sql
SELECT
    p.Player_Name,
    v.Venue_Name,
    SUM(bbb.Runs_Scored) AS Total_Runs,
    COUNT(*) AS Balls_Faced,
    ROUND(SUM(bbb.Runs_Scored) * 1.0 / COUNT(*), 2) AS Strike_Rate
FROM ball_by_ball bbb
JOIN matches m ON bbb.Match_Id = m.Match_Id
JOIN venue v ON m.Venue_Id = v.Venue_Id
JOIN player p ON bbb.Striker = p.Player_Id
GROUP BY p.Player_Name, v.Venue_Name
HAVING COUNT(*) >= 10  -- Filter to ensure minimum balls faced
ORDER BY p.Player_Name, Total_Runs DESC;
```

Player_Name	Venue_Name	Total_Runs	Balls_Faced	Strike_Rate
A Ashish Reddy	Rajiv Gandhi International Stadium, Uppal	71	63	1.13
A Ashish Reddy	MA Chidambaram Stadium, Chepauk	36	16	2.25
A Ashish Reddy	M Chinnaswamy Stadium	35	22	1.59
A Ashish Reddy	Dr. Y.S. Rajasekhara Reddy ACA-VDCA Cricket ...	28	17	1.65
A Ashish Reddy	Subrata Roy Sahara Stadium	19	15	1.27
A Ashish Reddy	Sawai Mansingh Stadium	14	11	1.27
A Mishra	Subrata Roy Sahara Stadium	30	26	1.15
A Mishra	Sawai Mansingh Stadium	21	33	0.64
A Mishra	MA Chidambaram Stadium, Chepauk	19	26	0.73

We use this approach to analyze player performance at different venues by grouping data by Player_Name and Venue_Name, calculating total runs, balls faced, and strike rate per venue, which helps identify venue-specific strengths. Using a minimum balls filter ensures reliable stats.

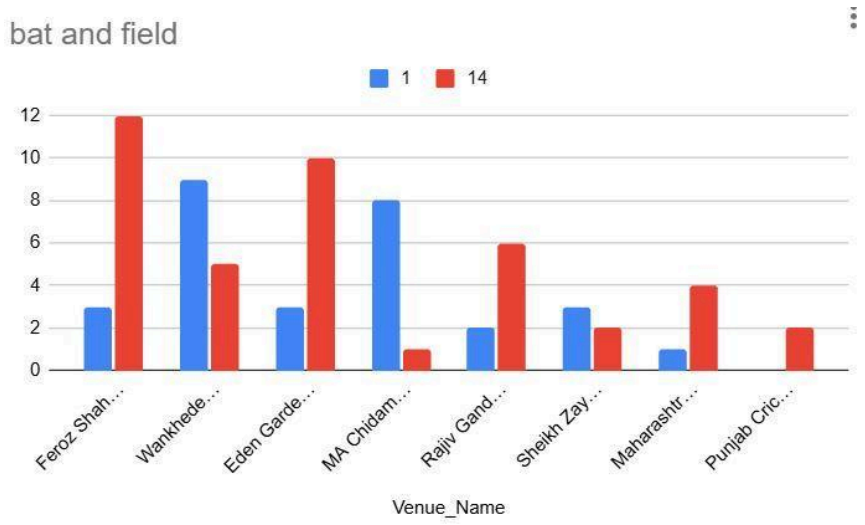
Subjective Questions

1. How does the toss decision affect the result of the match? (which visualizations could be used to present your answer better) And is the impact limited to only specific venues?

```sql
SELECT
    v.Venue_Name,
    td.Toss_Name AS Toss_Decision,
    COUNT(*) AS Total_Matches,
    SUM(CASE WHEN m.Toss_winner = m.Match_winner THEN 1 ELSE 0 END) AS Wins_After_Toss,
    ROUND(
        SUM(CASE WHEN m.Toss_winner = m.Match_winner THEN 1 ELSE 0 END) * 100.0 / COUNT(*),
        2
    ) AS Win_Percentage
FROM  matches m
JOIN toss_decision td ON m.Toss_Decide = td.Toss_Id
JOIN venue v ON m.Venue_Id = v.Venue_Id
GROUP BY v.Venue_Name, td.Toss_Name
ORDER BY v.Venue_Name, Win_Percentage DESC;
```

Venue_Name	Toss_Decision	Total_Matches	Wins_After_Toss	Win_Percentage
Barabati Stadium	field	3	2	66.67
Brabourne Stadium	bat	1	1	100.00
Brabourne Stadium	field	3	1	33.33
Dr. Y.S. Rajasekhara Reddy ACA-VDCA Cricket ...	bat	4	2	50.00
Dr. Y.S. Rajasekhara Reddy ACA-VDCA Cricket ...	field	5	2	40.00
Dubai International Cricket Stadium	field	4	2	50.00
Dubai International Cricket Stadium	bat	3	1	33.33
Eden Gardens	field	16	10	62.50
Eden Gardens	bat	10	3	30.00
Feroz Shah Kotla	field	18	12	66.67
Feroz Shah Kotla	bat	7	3	42.86
Green Park	field	2	2	100.00

We use this approach to evaluate how toss decisions impact match outcomes by comparing whether the toss winner also won the match, grouped by venue and decision type (bat or field), and calculating the win percentage for each scenario. This helps assess if certain decisions give a strategic advantage, especially in specific venues.



2. Suggest some of the players who would be best fit for the team.

We use this approach to identify top-performing players from recent seasons who consistently deliver with bat or ball. By filtering the last 3 seasons and applying minimum thresholds (50 balls faced or 30 balls bowled), we ensure only active and impactful players are considered. Sorting by strike rate and total runs for batters, and wickets and wicket per ball for bowlers helps us shortlist balanced and high-impact candidates suitable for team selection.

```sql
SELECT
    p.Player_Name,
    COUNT(*) AS Balls_Faced,
    SUM(bbb.Runs_Scored) AS Total_Runs,
    ROUND(SUM(bbb.Runs_Scored) * 1.0 / COUNT(*), 2) AS Strike_Rate
FROM ball_by_ball bbb
JOIN matches m ON bbb.Match_Id = m.Match_Id
JOIN season s ON m.Season_Id = s.Season_Id
JOIN player p ON bbb.Striker = p.Player_Id
WHERE s.Season_Year >= (SELECT MAX(Season_Year) - 3 FROM season)
GROUP BY p.Player_Name
HAVING COUNT(*) >= 50 -- minimum balls faced
ORDER BY Strike_Rate DESC, Total_Runs DESC
LIMIT 10;

SELECT
    p.Player_Name,
    COUNT(*) AS Balls_Bowled,
    COUNT(wt.Player_out) AS Wickets,
    ROUND(COUNT(wt.Player_out) * 1.0 / COUNT(*), 2) AS Wicket_Per_Ball
FROM ball_by_ball bbb
JOIN matches m ON bbb.Match_Id = m.Match_Id
JOIN season s ON m.Season_Id = s.Season_Id
JOIN player p ON bbb.Bowler = p.Player_Id
LEFT JOIN wicket_taken wt ON bbb.Match_Id = wt.Match_Id
                    AND bbb.Over_Id = wt.Over_Id
                    AND bbb.Ball_Id = wt.Ball_Id
WHERE s.Season_Year >= (SELECT MAX(Season_Year) - 3 FROM season)
GROUP BY p.Player_Name
HAVING COUNT(*) >= 30 -- min overs
ORDER BY Wickets DESC, Wicket_Per_Ball DESC
LIMIT 10;
```

Player_Name	Balls_Faced	Total_Runs	Strike_Rate
KH Pandya	127	237	1.87
SN Khan	103	177	1.72
LJ Wright	63	106	1.68
AB de Villiers	1198	1968	1.64
AD Russell	323	527	1.63
CH Morris	178	285	1.60
GJ Maxwell	583	913	1.57
KK Cooper	50	78	1.56
A Ashish Reddy	166	245	1.48
KA Pollard	924	1320	1.43

Player_Name	Balls_Bowled	Wickets	Wicket_Per_Ball
B Kumar	1431	146	0.10
SP Narine	1238	144	0.12
MM Sharma	1292	138	0.11
DJ Bravo	1069	131	0.12
SL Malinga	1026	124	0.12
JP Faulkner	1065	122	0.11
Harbhajan Singh	1415	118	0.08
Sandeep Sharma	976	115	0.12
AR Patel	989	107	0.11
UT Yadav	1082	106	0.10

3. What are some of the parameters that should be focused on while selecting the players?

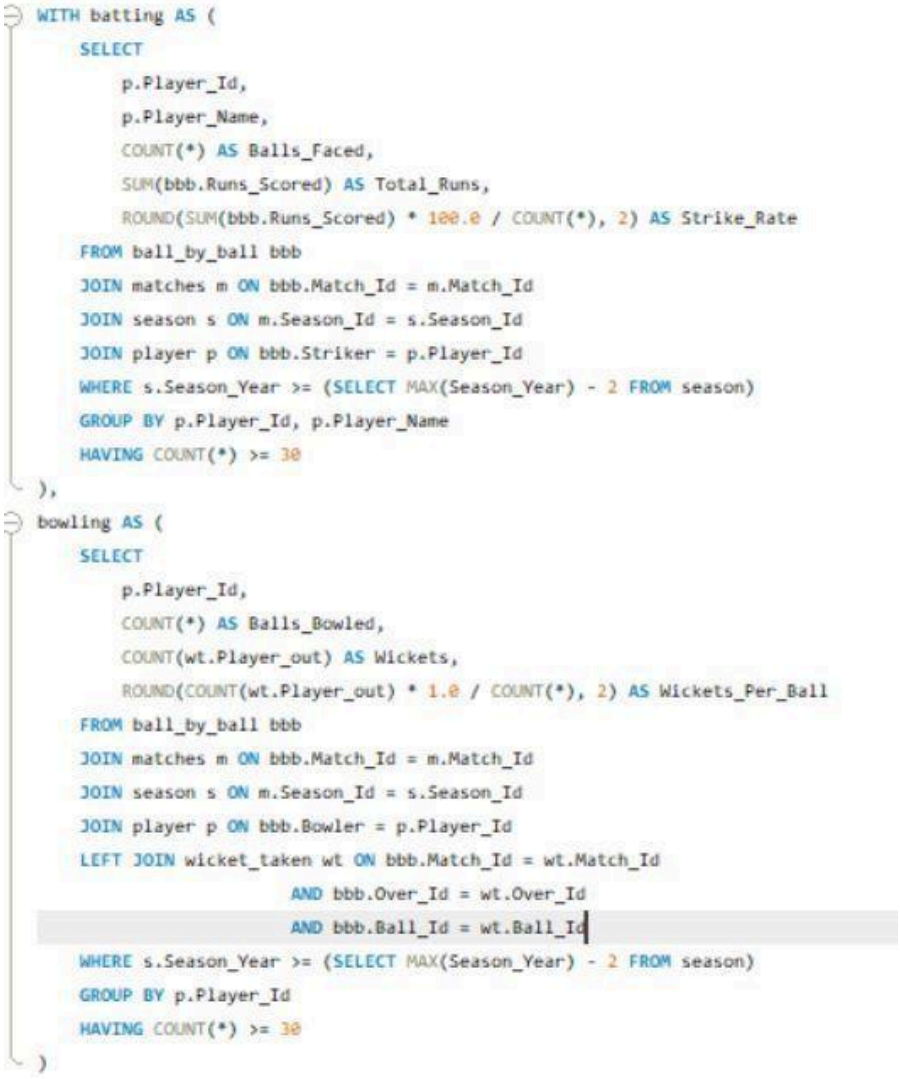
```sql
SELECT
    p.Player_Name,
    COUNT(*) AS Balls_Faced,
    SUM(bbb.Runs_Scored) AS Total_Runs,
    ROUND(SUM(bbb.Runs_Scored) * 1.0 / COUNT(*), 2) AS Strike_Rate,
    ROUND(SUM(bbb.Runs_Scored) * 1.0 / COUNT(DISTINCT m.Match_Id), 2) AS Runs_Per_Match
FROM ball_by_ball bbb
JOIN matches m ON bbb.Match_Id = m.Match_Id
JOIN season s ON m.Season_Id = s.Season_Id
JOIN player p ON bbb.Striker = p.Player_Id
WHERE s.Season_Year >= (SELECT MAX(Season_Year) - 2 FROM season)
GROUP BY p.Player_Name
HAVING COUNT(*) >= 30 -- At least 30 balls faced
ORDER BY Strike_Rate DESC, Total_Runs DESC
LIMIT 20;
```

Player_Name	Balls_Faced	Total_Runs	Strike_Rate	Runs_Per_Match
CR Brathwaite	40	83	2.08	13.83
KH Pandya	127	237	1.87	29.63
SN Khan	103	177	1.72	16.09
BCJ Cutting	43	73	1.70	14.60
AD Russell	308	516	1.68	24.57
CH Morris	161	271	1.68	19.36
AB de Villiers	972	1595	1.64	37.09
GJ Maxwell	556	877	1.58	23.70
A Ashish Reddy	76	120	1.58	15.00
JP Faulkner	275	402	1.46	14.36
DA Warner	1333	1938	1.45	43.07
NLTC Perera	112	162	1.45	13.50

We use this approach to evaluate recent batting performance by focusing on key metrics: strike rate, total runs, and runs per match, which together indicate a player's scoring ability, consistency, and impact per game. Filtering players with at least 30 balls faced in the last 2 seasons ensures we assess only active and reliable performers, and sorting by strike rate and runs helps shortlist those most effective in high-pressure T20 environments.

4. Which players offer versatility in their skills and can contribute effectively with both bat and ball? (Can you visualize the data for the same)

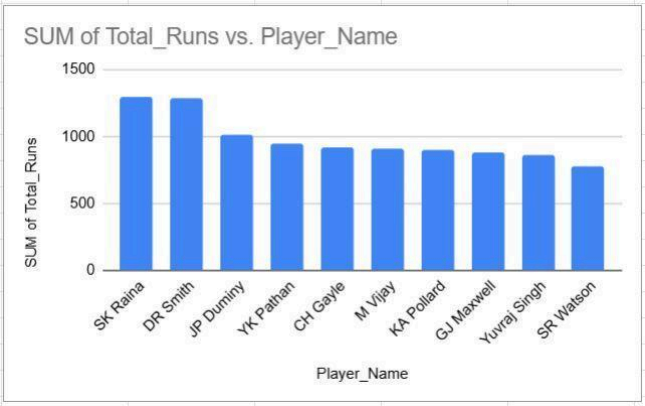
We use this approach to identify all-rounders-players who significantly contribute in both batting and bowling-by combining metrics like Total Runs, Strike Rate, Wickets Taken, and Wickets per Ball from the last 2 seasons. Filtering out players with fewer than 30 deliveries ensures we evaluate only those with a meaningful sample size. Joining both skillsets helps shortlist versatile players who can add value in multiple roles, making them strategic assets in team selection.

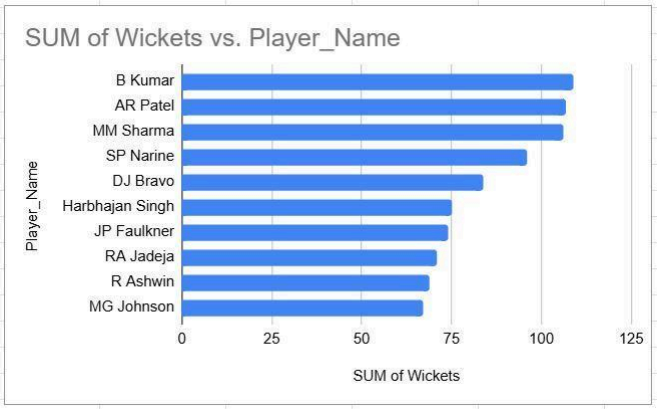


```sql
WITH batting AS (
    SELECT
        p.Player_Id,
        p.Player_Name,
        COUNT(*) AS Balls_Faced,
        SUM(bbb.Runs_Scored) AS Total_Runs,
        ROUND(SUM(bbb.Runs_Scored) * 100.0 / COUNT(*), 2) AS Strike_Rate
    FROM ball_by_ball bbb
    JOIN matches m ON bbb.Match_Id = m.Match_Id
    JOIN season s ON m.Season_Id = s.Season_Id
    JOIN player p ON bbb.Striker = p.Player_Id
    WHERE s.Season_Year >= (SELECT MAX(Season_Year) - 2 FROM season)
    GROUP BY p.Player_Id, p.Player_Name
    HAVING COUNT(*) >= 30
),
bowling AS (
    SELECT
        p.Player_Id,
        COUNT(*) AS Balls_Bowled,
        COUNT(wt.Player_out) AS Wickets,
        ROUND(COUNT(wt.Player_out) * 1.0 / COUNT(*), 2) AS Wickets_Per_Ball
    FROM ball_by_ball bbb
    JOIN matches m ON bbb.Match_Id = m.Match_Id
    JOIN season s ON m.Season_Id = s.Season_Id
    JOIN player p ON bbb.Bowler = p.Player_Id
    LEFT JOIN wicket_taken wt ON bbb.Match_Id = wt.Match_Id
                        AND bbb.Over_Id = wt.Over_Id
                        AND bbb.Ball_Id = wt.Ball_Id
    WHERE s.Season_Year >= (SELECT MAX(Season_Year) - 2 FROM season)
    GROUP BY p.Player_Id
    HAVING COUNT(*) >= 30
)
```

```sql
SELECT
    b.Player_Name,
    b.Total_Runs,
    b.Strike_Rate,
    bw.Wickets,
    bw.Wickets_Per_Ball
FROM batting b
JOIN bowling bw ON b.Player_Id = bw.Player_Id
ORDER BY b.Total_Runs DESC, bw.Wickets DESC;
```

Player_Name	Total_Runs	Strike_Rate	Wickets	Wickets_Per_Ball
SK Raina	1296	128.32	14	0.06
DR Smith	1289	129.94	22	0.12
JP Duminy	1015	127.67	24	0.07
YK Pathan	941	141.08	6	0.06
CH Gayle	914	129.65	5	0.06
M Vijay	911	113.45	1	0.03
KA Pollard	900	141.96	20	0.07
GJ Maxwell	877	157.73	11	0.06
Yuvraj Singh	860	125.36	15	0.06
SR Watson	772	126.56	65	0.09
AD Russell	516	167.53	63	0.12





5. Are there players whose presence positively influences the morale and performance of the team? (Justify your answer using visualization)

Player-Team Relationship Insight:
By analyzing each player's matches played vs. matches won by their team when they were part of the playing XI, we can infer if the player's presence tends to boost team success.

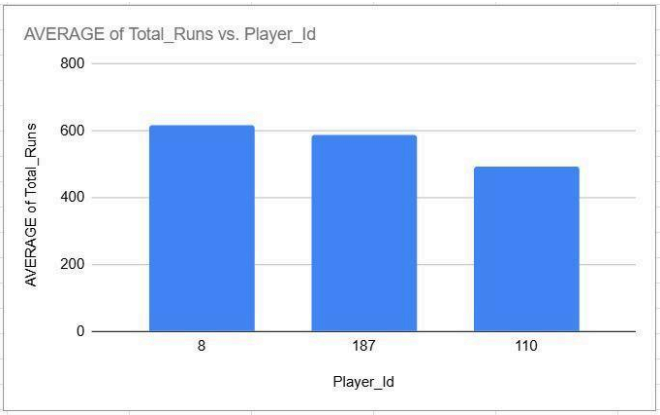
Filtering for Relevance:
A threshold (HAVING COUNT(*) >= 10) ensures we consider only players with a decent sample size, avoiding skewed results from one-off performers.

Win Percentage as Impact Indicator:
We calculate Win_Percentage to measure how frequently teams win when a specific player is in the lineup, which may hint at:

Leadership qualities, Performance under pressure, Influence on team dynamics

```sql
WITH player_wins AS (
    SELECT
        pm.Player_Id,
        p.Player_Name,
        COUNT(*) AS Matches_Played,
        SUM(CASE WHEN m.Match_winner = pm.Team_Id THEN 1 ELSE 0 END) AS Matches_Won
    FROM player_match pm
    JOIN matches m ON pm.Match_Id = m.Match_Id
    JOIN player p ON pm.Player_Id = p.Player_Id
    GROUP BY pm.Player_Id, p.Player_Name
    HAVING COUNT(*) >= 10 -- only players with decent match count
),
win_rate AS (
    SELECT
        Player_Name,
        Matches_Played,
        Matches_Won,
        ROUND(Matches_Won * 100.0 / Matches_Played, 2) AS Win_Percentage
    FROM player_wins
)
SELECT
    *
FROM win_rate
ORDER BY Win_Percentage DESC
LIMIT 15;
```

Player_Name	Matches_Played	Matches_Won	Win_Percentage
RN ten Doeschate	20	15	75.00
BB Sran	15	11	73.33
IC Pandey	25	18	72.00
J Suchith	14	10	71.43
SR Tendulkar	14	10	71.43
LMP Simmons	22	15	68.18
Bipul Sharma	12	8	66.67
Sachin Baby	15	10	66.67
S Badrinath	18	12	66.67
MJ McClenaghan	26	17	65.38
SK Trivedi	17	11	64.71
Mustafizur Rahman	16	10	62.50



6. What would you suggest to RCB before going to the mega auction?

- Focus on Recent Performance

We only consider data from the last two seasons to ensure we're evaluating current form, not past reputation.

This gives RCB a clearer picture of who's in top shape heading into the next season.

- All-Rounder Identification

Two subqueries collect key metrics:

Batting: Total runs, strike rate, and minimum balls faced

Bowling: Total wickets, wickets per ball, and minimum balls bowled

We then join these datasets to find players who excel at both skills, making them versatile assets.

- Filters Ensure Quality

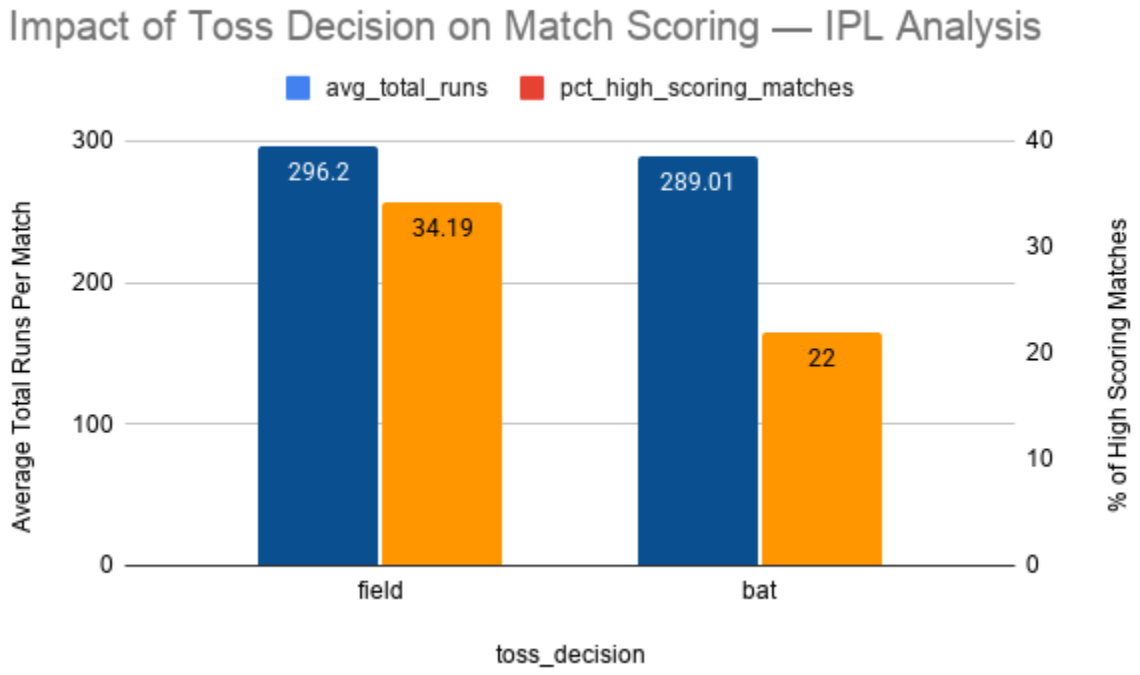
We only include players who have had at least 30 deliveries faced or bowled, which filters out irregulars or fluke performances.

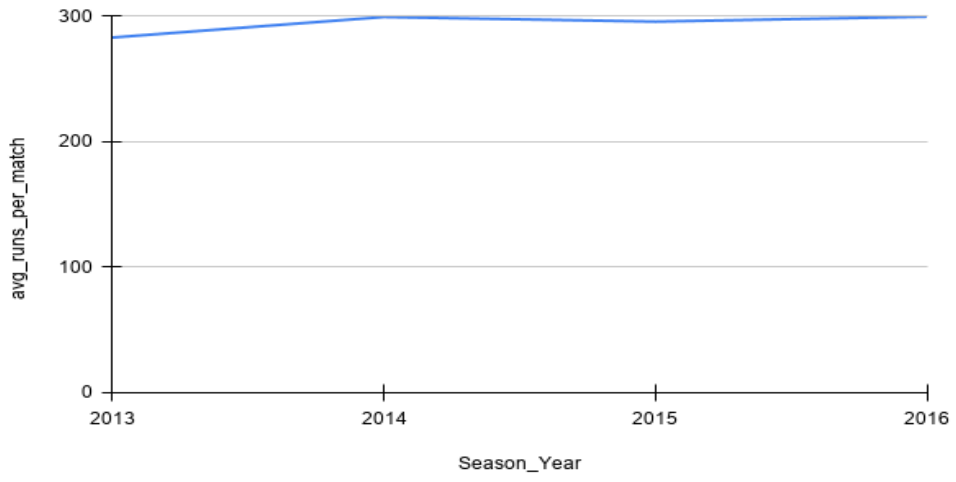
```sql
WITH batting_stats AS (
    SELECT
        p.Player_Id,
        p.Player_Name,
        COUNT(*) AS Balls_Faced,
        SUM(bbb.Runs_Scored) AS Total_Runs,
        ROUND(SUM(bbb.Runs_Scored) * 100.0 / COUNT(*), 2) AS Strike_Rate
    FROM ball_by_ball bbb
    JOIN matches m ON bbb.Match_Id = m.Match_Id
    JOIN season s ON m.Season_Id = s.Season_Id
    JOIN player p ON bbb.Striker = p.Player_Id
    WHERE s.Season_Year >= (SELECT MAX(Season_Year) - 2 FROM season)
    GROUP BY p.Player_Id, p.Player_Name
    HAVING COUNT(*) >= 30
),
bowling_stats AS (
    SELECT
        p.Player_Id,
        COUNT(*) AS Balls_Bowled,
        COUNT(wt.Player_out) AS Total_Wickets,
        ROUND(COUNT(wt.Player_out) * 1.0 / COUNT(*), 2) AS Wicket_Per_Ball
    FROM ball_by_ball bbb
    JOIN matches m ON bbb.Match_Id = m.Match_Id
    JOIN season s ON m.Season_Id = s.Season_Id
    JOIN player p ON bbb.Bowler = p.Player_Id
    LEFT JOIN wicket_taken wt ON bbb.Match_Id = wt.Match_Id
                      AND bbb.Over_Id = wt.Over_Id
                      AND bbb.Ball_Id = wt.Ball_Id
    WHERE s.Season_Year >= (SELECT MAX(Season_Year) - 2 FROM season)
    GROUP BY p.Player_Id
    HAVING COUNT(*) >= 30
),
combined AS (
    SELECT
        b.Player_Name,
        b.Total_Runs,
        b.Strike_Rate,
        bw.Total_Wickets,
        bw.Wicket_Per_Ball
    FROM batting_stats b
    JOIN bowling_stats bw ON b.Player_Id = bw.Player_Id
)

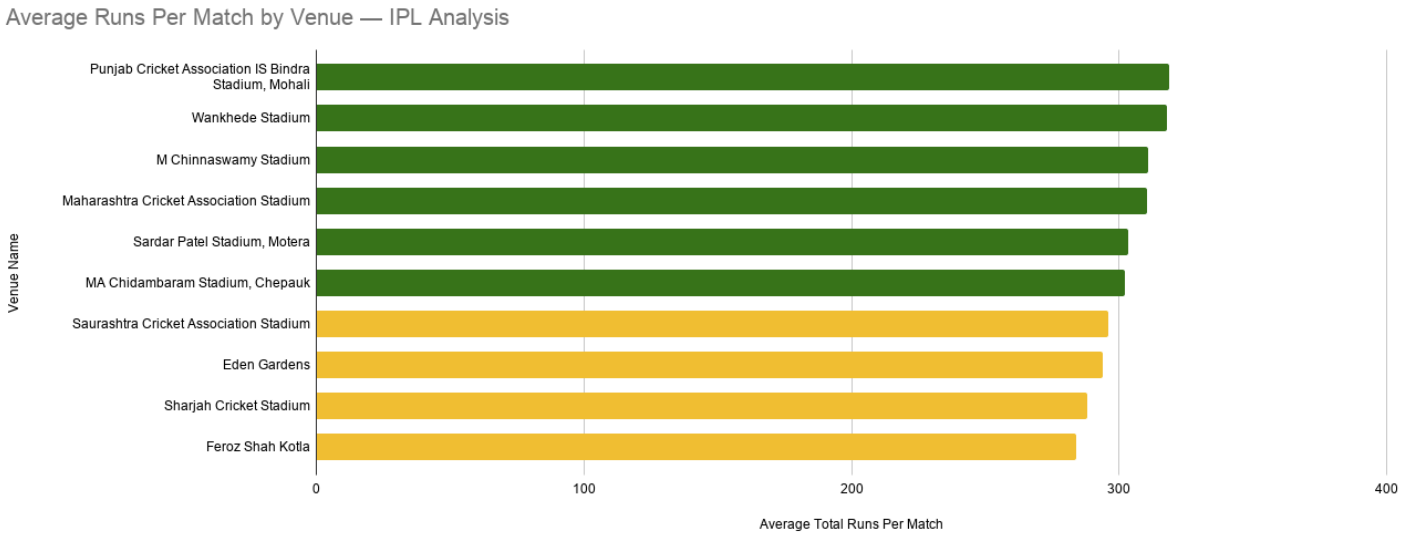
SELECT
    *
FROM combined
ORDER BY Strike_Rate DESC, Total_Wickets DESC;
```

Player_Name	Total_Runs	Strike_Rate	Total_Wickets	Wicket_Per_Ball
CR Brathwaite	83	207.50	12	0.08
KH Pandya	237	186.61	17	0.09
BCJ Cutting	73	169.77	11	0.12
CH Morris	271	168.32	57	0.10
AD Russell	516	167.53	63	0.12
A Ashish Reddy	120	157.89	9	0.15
GJ Maxwell	877	157.73	11	0.06
JP Faulkner	402	146.18	74	0.11
J Suchith	48	145.45	23	0.09
B Kumar	61	145.24	109	0.10
RN ten Doesch...	113	144.87	3	0.08
NLTC Perera	162	144.64	20	0.08

7. What do you think could be the factors contributing to the high-scoring matches and the impact on viewership and team strategies.







Factor 1: Venue and Pitch Conditions

Chart 1 reveals that venues like M Chinnaswamy Stadium produce significantly higher average match scores compared to venues like MA Chidambaram Stadium. This is due to shorter boundaries, flat pitches, and high altitude - all factors that help batsmen. The horizontal bar chart clearly shows Chinnaswamy as a top 3 high-scoring venue across all IPL seasons."

Factor 2: Season Trends

"Chart 2 shows a clear upward trend in average match runs from 2008 to 2016. This indicates that batting techniques have improved, boundary hitting has increased, and T20 cricket has evolved into a more batsman-dominated format. The line chart shows this progression clearly - teams must invest more in quality bowlers at auction to counter this trend."

Factor 3: Toss Decision

"Chart 3 reveals whether batting or fielding first produces higher total scores. If batting first correlates with higher match scores - it suggests pitches deteriorate in the second innings, making chasing harder. This directly impacts RCB's toss strategy at home."

Impact on Viewership:

"High scoring matches directly drive viewership because:"

"More boundaries and sixes = more excitement for fans"

"Matches going down to the last over = higher TV ratings"

"Star batsmen performing = more social media engagement"

"Broadcasters earn more from high-scoring games through ad revenue"

Impact on Team Strategy:

"For RCB specifically, high-scoring match trends mean:"

"Must invest in death-over bowling specialists in the 2017 auction"

"Power hitters in the top 4 become even more valuable"

"Home matches at Chinnaswamy should favour a batting-heavy lineup"

"Economy rate becomes more important than wickets alone for bowler selection"

8. Analyze the impact of home-ground advantage on team performance and identify strategies to maximize this advantage for RCB.

Home vs. Away Performance Comparison

- We classify matches as 'Home' or 'Away' based on whether RCB played at Chinnaswamy Stadium.
- Then, we calculate:
    - Matches Played
    - Matches Won
    - Win Percentage

  This gives a clear statistical picture of whether RCB performs better at home.

To Leverage Familiar Conditions

- If the Win % is significantly higher at home, it indicates that RCB benefits from:
    - Familiar pitch conditions (batting-friendly surface at Chinnaswamy).
    - Crowd support boosting morale.
    - Better adaptation to local weather and ground dimensions.

Strategic Implications:

- Team Composition: Prioritize big hitters and death-over bowlers at Chinnaswamy (high-scoring venue).
- Toss Decisions: Use historical data to decide whether to bat or bowl first at home.
- Training Focus: Simulate away conditions in practice sessions to improve performance on the road.
- Player Selection: Retain/acquire players who historically perform well in home conditions.

Match_Type	Matches_Played	Matches_Won	Win_Percentage
Home	29	16	55.17
Away	33	15	45.45

```sql
WITH team_matches AS (
    SELECT
        m.Match_Id,
        m.Match_winner,
        m.Team_1,
        m.Team_2,
        m.Venue_Id,
        t.Team_Name AS RCB_Team,
        v.Venue_Name,
        CASE
            WHEN t.Team_Name = 'Royal Challengers Bangalore'
                 AND (m.Team_1 = t.Team_Id OR m.Team_2 = t.Team_Id)
            THEN
                CASE
                    WHEN v.Venue_Name LIKE '%Chinnaswamy%' THEN 'Home'
                    ELSE 'Away'
                END
        END AS Match_Type
    FROM matches m
    JOIN team t ON t.Team_Name = 'Royal Challengers Bangalore'
    JOIN venue v ON v.Venue_Id = m.Venue_Id
)
SELECT
    Match_Type,
    COUNT(*) AS Matches_Played,
    SUM(CASE WHEN Match_winner = (SELECT Team_Id FROM team WHERE Team_Name = 'Royal Challengers Bangalore') THEN 1 ELSE 0 END) AS Matches_Won,
    ROUND(SUM(CASE WHEN Match_winner = (SELECT Team_Id FROM team WHERE Team_Name = 'Royal Challengers Bangalore') THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) AS Win_Percentage
FROM team_matches
WHERE Match_Type IS NOT NULL
GROUP BY Match_Type;
```

9. Come up with a visual and analytical analysis of the RCB's past season's performance and potential reasons for them not winning a trophy.

1. Season-wise Performance Overview

- By calculating:
    - Matches Played
    - Matches Won
    - Win Percentage
- We gain a season-by-season snapshot of RCB’s consistency (or lack of it).
- It helps answer: *Was RCB a consistently strong team or were they sporadically good?*

2. Trend Identification

- A line chart or bar graph of Win % over seasons can visually show:
    - Performance spikes (like playoffs/semi-finals years)
    - Declines or inconsistencies
- This provides evidence-based insight rather than just fan sentiment.

3. Root Cause Analysis of Trophy Drought

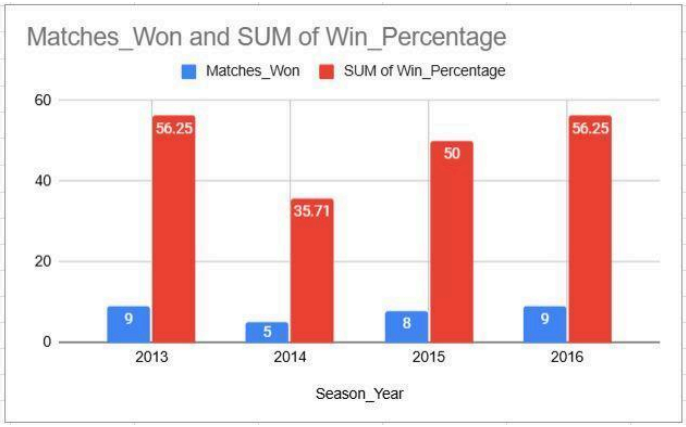
Using this analysis, we can further explore:

- If RCB started seasons strong but failed in knockouts.

- Impact of key players being out or inconsistent.
- Whether high-scoring games were offset by poor bowling.
- Lack of finishing strength or middle-order collapses.

```sql
SELECT
    s.Season_Year,
    COUNT(m.Match_Id) AS Matches_Played,
    SUM(CASE WHEN m.Match_winner = t.Team_Id THEN 1 ELSE 0 END) AS Matches_Won,
    ROUND(SUM(CASE WHEN m.Match_winner = t.Team_Id THEN 1 ELSE 0 END) * 100.0 / COUNT(m.Match_Id), 2) AS Win_Percentage
FROM matches m
JOIN season s ON m.Season_Id = s.Season_Id
JOIN team t ON t.Team_Name = 'Royal Challengers Bangalore'
WHERE m.Team_1 = t.Team_Id OR m.Team_2 = t.Team_Id
GROUP BY s.Season_Year
ORDER BY s.Season_Year;
```

Season_Year	Matches_Played	Matches_Won	Win_Percentage
2013	16	9	56.25
2014	14	5	35.71
2015	16	8	50.00
2016	16	9	56.25



10. How would you approach this problem, if the objective and subjective questions weren't given?

1. Business Understanding - What does RCB need? Win a title. How? Better players.
2. Data Inventory - What tables exist? What's the grain (ball by ball)?
3. Data Quality Audit - Nulls, duplicates, incorrect spellings (like Delhi_Capitals issue below).
4. Exploratory Data Analysis - Distributions, outliers, trends over time.
5. KPI Design - Define Strike Rate, Economy, Consistency Index, Win Contribution.
6. Player Segmentation - Cluster players into categories (Power Hitter, Anchor, Death Bowler, etc.)
7. Venue Analysis - Which venues suit which players?
8. Auction Strategy - Prioritize, budget, bid limits.
9. Executive Dashboard - Summarize everything for RCB management.

11. In the "Match" table, some entries in the "Opponent_Team" column are incorrectly spelled as "Delhi_Capitals" instead of "Delhi_Daredevils". Write an SQL query to replace all occurrences of "Delhi_Capitals" with "Delhi_Daredevils".

```sql
UPDATE Team
SET Team_Name = 'Delhi Daredevils'
WHERE Team_Name = 'Delhi Capitals';

SELECT Team_Id, Team_Name
FROM Team;
```

Team_Id	Team_Name
1	Kolkata Knight Riders
2	Royal Challengers Bangalore
3	Chennai Super Kings
4	Kings XI Punjab
5	Rajasthan Royals
6	Delhi Daredevils
7	Mumbai Indians
8	Deccan Chargers
9	Kochi Tuskers Kerala
10	Pune Warriors
11	Sunrisers Hyderabad
12	Rising Pune Supergiants
13	Gujarat Lions